第3课 GPIO引脚输出设置

1.GPIO引脚介绍

GPIO是通用输入或输出端口的简称，是电子设备主板上的一组引脚，这些引脚可以发送或接收电信号。将引脚与外部硬件设备连接，可实现与外部通讯、控制外部硬件或者采集外部硬件数据的功能。

2. 输出介绍

输出是当需要控制引脚电平的高低时需要用到的功能，高低电平可以用数字1和0表示。比如控制LED灯发光，需要在连接LED的引脚处，设置高电平状态，与另一端低电平形成电压差，产生电流，使LED灯放光。

3. 树莓派4B GPIO引脚介绍

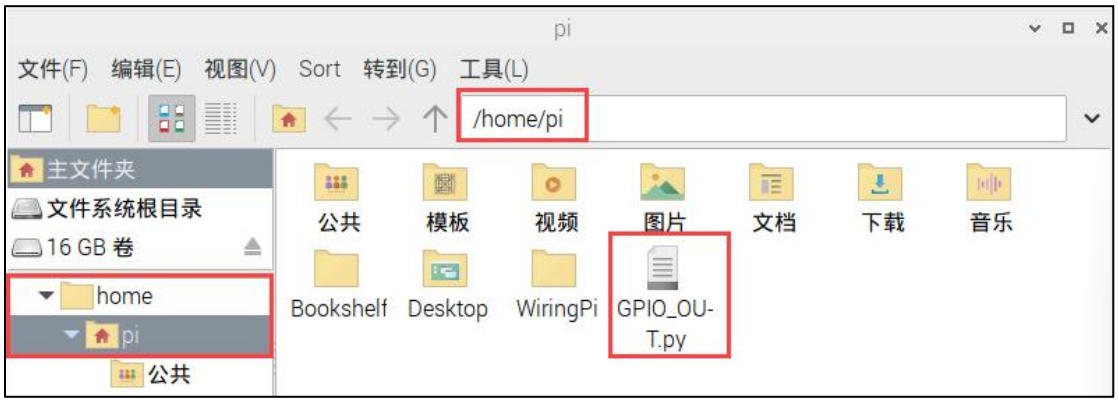
树莓派4B开发板上有一个40 Pin的接口，可以方便使用众多的扩展板卡。这些GPIO的控制如读，写，中断, PWM等都可以通过RPi.GPIO库来进行。

引脚分布如图所示：

wiringPi 编码	BCM 编码	功能名	物理引脚 BOARD 编码		功能名	BCM 编码	wiringPi 编码
		3.3V	1	2	5V		
8	2	SDA.1	3	4	5V		
9	3	SCL.1	5	6	GND		
7	4	GPIO.7	7	8	TXD	14	15
		GND	9	10	RXD	15	16
0	17	GPIO.0	11	12	GPIO.1	18	1
2	27	GPIO.2	13	14	GND		
3	22	GPIO.3	15	16	GPIO.4	23	4
		3.3V	17	18	GPIO.5	24	5
12	10	MOSI	19	20	GND		
13	9	MISO	21	22	GPIO.6	25	6
14	11	SCLK	23	24	CE0	8	10
		GND	25	26	CE1	7	11
30	0	SDA.0	27	28	SCL.0	1	31
21	5	GPIO.21	29	30	GND		
22	6	GPIO.22	31	32	GPIO.26	12	26
23	13	GPIO.23	33	34	GND		
24	19	GPIO.24	35	36	GPIO.27	16	27
25	26	GPIO.25	37	38	GPIO.28	20	28
		GND	39	40	GPIO.29	21	29

4. 输出设置

1) 把同目录下的“**GPIO_OUT.py**”程序文件导入进主板系统中，放置文件在主目录下。

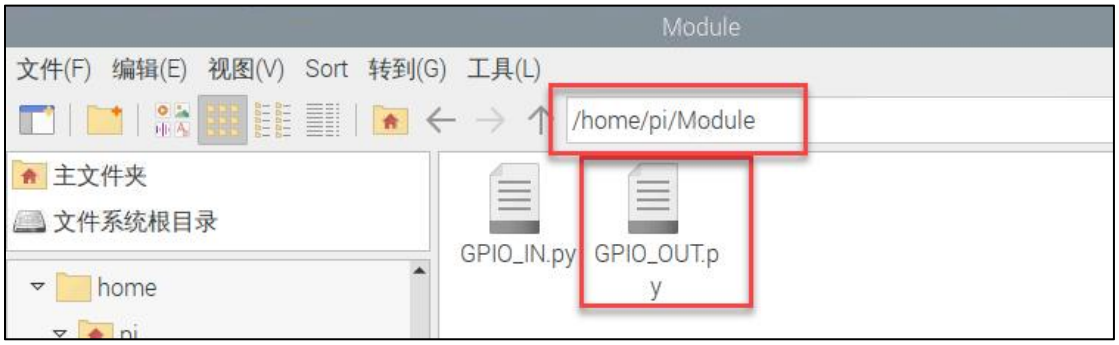


2) 双击系统桌面图标 ，输入指令“**python3 GPIO_OUT.py**”，按下回车执行程序。

```
pi@raspberrypi:~ $ python3 GPIO_OUT.py
```

3) 程序执行后，使用万用表测量 37 号引脚会在高低电平之间不断切换，例如将引脚接上 3.3V 的电压后，会看都电压在 3.3V 和 0V 间变化。

4) 为了方便后期查看，我们可将此程序剪切到“**home\pi\Module**”文件夹目录下(用户需自行在主目录下创建Module文件夹)。



5. 程序分析

◆ **导入库文件**

导入GPIO库文件和time库文件，该库文件已经预装在了系统当中，直接导入使用即可。

```
import RPi.GPIO as GPIO
import time
```

◆ **设置需要读取的引脚编号**

设置需要操作的引脚编号，程序默认为37号。

```
# Pin Definitions
output_pin = 37
```

◆ **取消警示消息提醒**

通过setwarnings函数设置是否开启警示消息，Flase为关闭，True为开启。

```
# Pin Setup:
GPIO.setwarnings(False)
```

◆ **设置GPIO模式**

设置GPIO的编码模式。

```
# Board pin-numbering scheme
GPIO.setmode(GPIO.BOARD)
```

使用setmode函数设置编码模式，模式一共有四种，分别是BOARD、BCM、CVM和TEGRA_SOC。

◆ **初始化引脚**

通过setup函数初始化引脚，其中“**output_pin**”参数为引脚编号，“**GPIO.OUT**”参数是设置该引脚为输出模式，“**initial**”参数为初始的电平状态。

```
# set pin as an input pin
GPIO.setup(output_pin, GPIO.OUT,initial=GPIO.HIGH)
```

◆ **设置输出**

通过output函数设置引脚的电平状态，“**GPIO.HIGH**”为高电平，“**GPIO.LOW**”为低电平。

```
while True:
        GPIO.output(output_pin, GPIO.HIGH)
        time.sleep(1)
        GPIO.output(output_pin, GPIO.LOW)
        time.sleep(1)
```